Tomasulo Algorithm Implementation

Mohammad Taufeeq Shaik

PS ID 2493846

1 Introduction

This assignment adds in order issue of one through four instructions per cycle to the supplied Tomasulo simulator. The delivered simulator supports register renaming, an in order reorder buffer, out of order execution, configurable functional units, memory operations and speculative branches. Results include dynamic instruction timings and final register and nonzero memory values.

The implementation passes 48 supplied or authored test and width combinations against a separate sequential ISA interpreter. An additional 200 deterministic randomized integer dependency runs pass. The comparison checks architectural results; timing is checked against structural invariants rather than an external timing oracle.

What Works

- Input selects issue width 1 through 4, default 1, along with reservation station counts, execution cycles, functional unit counts, memory cycles and ROB capacity.
- Issue stops at the first structural stall, captures sources before renaming the destination, and retires instructions in program order.
- Integer units are unpipelined; FP add and multiply units accept pipelined work. Loads use a single memory port or forward from the youngest matching older store.
- Eight tagged BTB entries carry one bit outcomes. Misprediction recovery clears younger work, rebuilds the RAT and resumes correct issue at cycle n plus 2.

Limitations and choices

The corrected execution core substantially replaces the starter routines rather than only wrapping their single issue loop; the original package is preserved under original/. Loads can access memory before older store addresses are resolved. If a newly resolved store aliases a younger load that has already accessed memory, younger instructions are squashed and reissued. Address faults are checked at retirement, so wrong path accesses can be discarded. Timing conventions are stated explicitly; no instructor cycle oracle is available.

2 Code Changes

The starter main.py hard codes machine parameters and register values. Its issue, execute, memory, writeback and commit functions pass mutable class objects returned by namedtuple factories rather than record instances. The delivered code/main.py supplies a command line entry point, and code/simulator.py replaces the fragile state and cycle logic with per instruction records. original/ retains the unmodified source and author comments for comparison.

2.1 State and input configuration

Initial Version

```
def rs_entry():
    rs_entry = namedtuple('rs_entry', 'busy, op, ...')
    temp = rs_entry
    return temp

# main.py uses fixed values such as:
size_ROB = 64
size_rs_int_adder = 4
```

My Version

```
@dataclass
class Hardware:
    # Independent dictionaries are created for each instance.
    rs: dict = field(default_factory=lambda:
        dict(integer=4, fp_add=3, fp_mul=2, ls=5))
    issue_width: int = 1
    writeback_width: int = 1
    commit_width: int = 1

@dataclass
class Record:
    tag: int
    pc: int
    ins: Instruction
    operands: list
    issue: int
    # EX, MEM, WB, COMMIT and squash state follow.
```

These are shortened excerpts; complete classes are in code/simulator.py. Monotonically increasing dynamic tags avoid stale tag reuse. The RAT maps architectural destinations to current ROB producers; operands carry either a pending tag or a value. Register R0 always reads zero and ignores writes.

The parser reads the assignment hardware rows, optional Issue width, ROB entries, register assignments, memory initialization and all listed opcodes. Input accepts mixed case and commas. Memory is indexed by byte address in 0 through 255; byte granularity also accepts the assignment example of an unaligned address.

2.2 Issue and cycle control

Initial Version

```
# issue.py fetches one instruction per call.
ins = instructions[PC.PC]
# main.py invokes issue once per cycle.
```

My Version

```
for _ in range(self.hw.issue_width):
    if not 0 <= self.pc < len(self.program):
        break
    ins = self.program[self.pc]
    if len(self.rob) >= self.hw.rob_entries:
        break
    # Check RS or load/store capacity; stop on a stall.
    # Capture operands, allocate a unique ROB record,
    # then rename the destination and advance PC.
    if ins.op in BRANCH:
        # Predict using the tagged eight entry BTB.
        break
```

This excerpt summarizes the complete issue method. A stalled instruction terminates the bundle, so a younger independent instruction cannot bypass it. Earlier instructions within the same bundle update the RAT before the next instruction captures its operands, preserving RAW and WAW relationships.

2.3 Execution memory and branch handling

Each cycle retires ready older work, broadcasts completed results, advances memory, completes and starts execution, then issues new work. An instruction cannot execute in its issue cycle, write back in its completion cycle, or retire in its writeback cycle. A broadcast can feed a waiting execution operand during the same cycle. Completed integer or address units restart in the following cycle; FP starts are limited by configured unit count each cycle.

The default single CDB broadcasts one result per cycle. Commit width defaults to issue width; optional Writeback width and Commit width fields permit other bandwidths. These explicit timing policies avoid assuming the sample report is a cycle exact oracle.

Stores update memory only at ROB head, occupy the memory port for the configured duration, and retire afterward. Loads choose the youngest matching older store for one cycle forwarding; unknown older store addresses do not stall a load. A later detected alias squashes and reissues younger instructions, with the same two cycle restart convention used for branch recovery. A recovery at the end of EX in cycle n removes all younger ROB, operand and execution state and rebuilds the RAT from surviving entries; no instructions issue in n or n plus 1.

Branch offsets default to words, following PC plus 4 plus offset shifted left by 2. The supplied starter instead divides its offset by four. Branch offsets = bytes explicitly enables that legacy interpretation; tests/starter_legacy.in adds this setting without altering the supplied instruction sequence.

3 Results

3.1 Test configuration

Tests were executed on October 20, 2026 using Python 3 on macOS. The simulator uses only the standard library and requires Python 3.9 or later. Every test runs at issue widths 1, 2, 3 and 4; writeback width stays 1 and commit width equals issue width.

Default parameter	Value
Integer RS EX units	4 RS 1 cycle 1 unit
FP add RS EX units	3 RS 4 cycles 1 unit
FP multiply RS EX units	2 RS 15 cycles 1 unit
Load store RS EX units	5 entries 1 cycle 1 unit
Memory and ROB	5 cycles 64 entries
Registers memory	32 integer and FP registers 256 bytes

Each .in file records any overrides. For example, pressure uses a two entry ROB and one integer station; renaming uses two integer and FP add units; memory_forward uses four memory cycles and a ten cycle multiplier; branch_recovery uses a three cycle integer unit. This allows both wide and constrained machines to be checked.

3.2 Measured completion cycles

Test	Width 1	Width 2	Width 3	Width 4
arithmetic	31	28	27	26
branch_recovery	17	17	17	17
fp_pipeline	14	14	14	14
kernel	68	55	54	54
loop	20	20	20	20
memory_forward	26	23	30	30
memory_replay	19	19	19	19
parallel	11	11	11	11
pressure	18	18	18	18
renaming	14	11	10	10
starter_legacy	44	38	36	35
wrong_path_fault	21	21	21	21

All 48 cases match the reference interpreter for every register and nonzero memory location. Wider issue does not always reduce cycles: the single result bus and resource contention can dominate. In memory_forward, earlier memory port occupation at widths 3 and 4 changes the schedule and raises total cycles. These values are measured outputs, not inferred speedups.

3.3 Representative instruction results

For arithmetic.in at width 4, the input initializes R1 to 5, R2 to 7, F1 to 2 and F2 to 3. Its eight instructions exercise integer RAW dependencies, FP dependence, multiplication and hardwired R0. The table below comes directly from the simulator JSON output.

Instruction	ISSUE	EX	MEM	WB	COMMIT
Add R3, R1, R2	1	2-2	-	3	4
Sub R4, R3, R1	1	3-3	-	4	5
Addi R5, R4, 10	1	4-4	-	5	6
Add.d F3, F1, F2	1	2-5	-	6	7
Sub.d F4, F3, F1	2	6-9	-	10	11
Mult.d F5, F4, F2	2	10-24	-	25	26
Addi R0, R0, 99	2	5-5	-	7	26
Add R6, R0, R5	3	6-6	-	8	26

Final nonzero integer values are R1 = 5, R2 = 7, R3 = 12, R4 = 7, R5 = 17 and R6 = 17. Final nonzero FP values are F1 = 2, F2 = 3, F3 = 5, F4 = 3 and F5 = 9. R0 remains zero and this case writes no memory. Full output includes every register, all dynamic instructions and nonzero memory values.

Branch and memory checks

branch_recovery deliberately issues a taken Beq with the cold not taken prediction. Wrong path instructions include a store. At all four widths, four instructions commit, one misprediction is recorded, R3 is 7, R4 is 9 and the wrong path store does not alter memory. loop exercises a backward branch, prediction learning and final exit recovery.

memory_forward waits for a multiply result that supplies an older store, forwards to a matching load, then verifies the second store and load. renaming tests a long latency writer followed by a younger writer to the same destination; readers receive the youngest producer. pressure tests that bundle issue stalls in order when ROB or RS capacity is exhausted.

Additional verification

The test runner asserts stage ordering, in order retirement, issue and bus bandwidth, functional unit launches and unpipelined occupancy. Branch checks verify the one cycle recovery gap. Fifty seeded integer programs, each with dependencies and four issue width settings, add 200 architectural checks. A dedicated unknown store alias test confirms actual replay, and a wrong path invalid address test confirms fault suppression. An independent CLI invocation, width 1 default and rejection of width 5 are also verified.

3.4 Reproduction and deliverables

Run the following commands from the extracted Tomasulo submission folder. Each result file records a dynamic instruction table, architectural state and cycle count. JSON companions preserve the complete machine configuration and records; summary.csv contains all 48 measured cycle totals.

```
python3 code/main.py tests/arithmetic.in --issue-width 4
python3 code/main.py tests/starter_legacy.in --issue-width 2
python3 run_tests.py
```

The simulator also accepts Issue width = 2 in the input itself. The optional command line flag overrides that input setting. Running code/main.py without arguments uses code/code.in, the supplied testbench with the explicit legacy byte offset setting.

What changed from the starter

The new entry point and execution core replace hard coded setup, class based mutable records, single issue dispatch, unsafe RAT value replacement, incomplete branch handling and incomplete final state output. The architecture and ISA are based on the supplied Tomasulo assignment and starter, but this is a substantial core rewrite. It is not a minimal diff limited to issue.py. The original files remain available under original/; code comparison excerpts retain their provenance.

Known limits

Architectural state is checked against an independent interpreter, but no instructor supplied cycle oracle is available. The sample report uses a different timing model, including wider writeback and commit, so its cycle numbers are not claimed as reproduced. Optimistic loads use memory dependence replay when an older unresolved store later aliases them. Speculative memory faults are deferred until retirement. Replay recovery uses a two cycle restart convention, which the assignment does not otherwise specify. The tested programs cover both cases, but they do not exhaust all legal programs or configurations.

References

[1] programming-tomasulo.pdf, supplied in o3-simulator.zip. Pages 1 and 2 define the processor and ISA; pages 3 and 4 define configurable issue width, output, tests and submission deliverables.

[2] Supplied tomasulo-python.zip, original code comments attributed to Qing, May 2017. Original source and testbench preserved under original/.

[3] Sample_Tomasulo_Assignment_Report.pdf, supplied report structure reference. The new report follows Introduction, What Works, Limitations, Code Changes and Results, with results from the delivered implementation.

Submit this package to its own GitHub repository and the Tomasulo assignment entry. The branch predictor report and code belong to the separate branch predictor repository.